

cardiac_core — Object Atlas

```
import cardiac_core as cc
```

The object map

Object	Meaning	You get it from
<code>Grid</code>	structured geometry — node counts + spacing	<code>cc.Grid(...)</code>
<code>ConductivityConfig</code>	tissue physics → diffusivity (the χ /Cm firewall)	<code>cc.ConductivityConfig.bidomain(...)</code>
<code>Stim</code>	a masked, timed stimulus (current or voltage clamp)	<code>cc.Stim.boundary(g, "left")</code>
<code>IonicModel</code>	cell electrophysiology (TTP06 / ORd / hiPSC)	a name string, or <code>TTP06Model(...)</code>
<code>CardiacSimulation</code>	the live simulation handle	<code>cc.monodomain(...)</code> / <code>bidomain</code> / <code>lbn</code>
<code>SimulationResult</code>	the run output + analysis hooks	<code>sim.run(...)</code>
<code>Fields</code> / <code>VectorField</code>	spatial field calculus on a result	<code>r.fields</code>
<code>SingleCellResult</code>	0-D action potential	<code>cc.single_cell(...)</code>
<code>Video</code> / <code>Gradient</code> / <code>VideoInfo</code>	render spec → rendered file	<code>cc.Video(r)</code> → <code>cc.render(...)</code>
<code>CardiacMeshData</code>	the on-disk mesh (<code>.npz</code>)	<code>cc.create_cardiac_mesh(...)</code>
<code>Distribution</code>	per-node random parameter spec	<code>cc.Distribution(...)</code>
<code>SimulationSnapshot</code>	one frame from a generator run	<code>sim.snapshots(...)</code>

The pipeline:

```
Grid + ConductivityConfig + Stim → monodomain() → CardiacSimulation → .run() → SimulationResult → .fields / .video()
```

1. `Grid`

```
g = cc.Grid(Nx, Ny, dx, dy=None, *, mask=None, boundary_mode="face_mirror",
            device="cpu", dtype=torch.float64)
```

Attributes

Access	Meaning
<code>g.Nx</code> , <code>g.Ny</code>	node counts (int)
<code>g.dx</code> , <code>g.dy</code>	spacing, cm (<code>dy</code>) defaults to <code>dx</code>)
<code>g.mask</code>	(<code>Nx</code> , <code>Ny</code>) bool tissue mask, or <code>None</code>
<code>g.boundary_mode</code>	ghost/mirror edge rule (<code>"face_mirror"</code>)
<code>g.device</code> , <code>g.dtype</code>	tensor device / float dtype
<code>g.Lx</code> , <code>g.Ly</code>	<i>property</i> — domain size cm : <code>dx*(Nx-1)</code> , <code>dy*(Ny-1)</code>
<code>g.coordinates</code>	<i>property</i> — (<code>x</code> , <code>y</code>) tensors (<code>Nx</code> , <code>Ny</code>) , <code>ij</code> indexing: <code>x[0,0]==0</code> , <code>x[-1,0]==Lx</code>
<code>g.n_dof</code>	<i>property</i> — active nodes: <code>mask.sum()</code> if masked, else <code>Nx*Ny</code>

Methods — none.

For an exact `Lx × Ly` **cm** domain: `Nx = round(Lx/dx) + 1` . A 1-D cable is `Ny=1` .

2. `ConductivityConfig`

Never constructed raw — use a classmethod.

Constructors

Call	Does
<code>cc.ConductivityConfig.isotropic(sigma, chi=1400.0, Cm=1.0)</code>	one effective σ
<code>cc.ConductivityConfig.bidomain(sigma_i, sigma_e, chi=1400.0, Cm=1.0)</code>	intra + extracellular — most physical
<code>cc.ConductivityConfig.anisotropic(sigma_l, sigma_t, fiber_angle, chi=1400.0, Cm=1.0)</code>	fibers (angle in radians)

Attributes

Access	Meaning
<code>cond.sigma_i</code> , <code>cond.sigma_e</code>	intra / extracellular conductivity, mS/cm
<code>cond.sigma_l</code> , <code>cond.sigma_t</code>	longitudinal / transverse (anisotropic)
<code>cond.sigma_iso</code>	the isotropic value
<code>cond.chi</code>	surface-to-volume, cm⁻¹
<code>cond.Cm</code>	capacitance, µF/cm²
<code>cond.fiber_angle</code>	fiber angle, radians
<code>cond.sigma_eff</code>	<i>property</i> — effective σ mS/cm ; scalar, but an <code>(xx, yy, xy)</code> 3-tuple for anisotropic
<code>cond.D_eff</code>	<i>property</i> — diffusivity cm²/ms = <code> sigma_eff /(chi*Cm)</code> ; same scalar-or-tuple shape

Methods

Call	Does
<code>cond.for_monodomain()</code>	emit Form A — <code>D=sigma_eff/chi</code> , <code>chi=1</code> , real <code>Cm</code>
<code>cond.for_bidomain()</code>	emit Form B for the bidomain factory
<code>cond.for_lbm()</code>	emit Form B with fully-scaled <code>D</code>

3. `Stim`

```
s = cc.Stim(mask, *, amplitude=None, clamp=None, start_time=0.0, duration=2.0,
           bcl=0.0, num_pulses=1, label="stim")
```

Constructors

Call	Does
<code>cc.Stim.boundary(grid, side, *, width=None, **kw)</code>	border strip — <code>"left"/"right"/"top"/"bottom"</code>
<code>cc.Stim.point(grid, center, *, radius=None, **kw)</code>	blob at an <code>(x, y)</code> cm point
<code>cc.Stim.center(grid, *, radius=None, **kw)</code>	blob at the domain centre
<code>cc.Stim.from_region(grid, region, **kw)</code>	from a <code>callable(x,y)->mask</code> or an <code>(Nx,Ny)</code> array
<code>cc.Stim.from_dict(d)</code>	rebuild from the normalized dict

Attributes

Access	Meaning
<code>s.mask</code>	<code>(Nx, Ny)</code> numpy bool
<code>s.mode</code>	<code>"current"</code> or <code>"clamp"</code> — inferred from which keyword you used
<code>s.amplitude</code>	µA/µF , negative = depolarizing (default -52)
<code>s.clamp</code>	clamp level mV , or <code>None</code>
<code>s.start_time</code> , <code>s.duration</code>	active window, ms
<code>s.bcl</code> , <code>s.num_pulses</code>	pacing train
<code>s.label</code>	free-text tag

Methods

Call	Does
<code>s.times()</code>	list the pulse onsets — <code>[t0 + k·bcl]</code>
<code>s.n_nodes()</code>	count the nodes the mask actually hits
<code>s.to_dict()</code>	lower to the dict engines/ <code>.npz</code> consume — current mode only (raises on a clamp)

```
cc.Stim.boundary(g, "left", amplitude=-52, bcl=1000, num_pulses=5) # CURRENT injection
cc.Stim.boundary(g, "left", clamp=-20, duration=50)               # VOLTAGE clamp, all 3 engines
```

4. `IonicModel`

Usually just a **name string**: `"ttp06"` (default), `"ord"` (**LBM only**), `"paci"`, `"phas13"`, `"mhas13"`.

```
from cardiac_core.ionic import TTP06Model, ORdModel, CellType
m = TTP06Model(cell_type=CellType.EPI) # ENDO (default) / EPI / M
```

Constructors

Call	Does
<code>TTP06Model.from_config(config, device=None, dtype=torch.float64, use_lut=False, base_cell_type=CellType.EPI)</code>	build from a <code>CellTypeConfig</code>

Attributes

Access	Meaning
<code>m.cell_type</code>	the <code>CellType</code> member — <code>ENDO</code> / <code>EPI</code> / <code>M</code>
<code>m.params</code>	the parameter object — maximal conductances live here
<code>m.params.GNa</code> , <code>.GK1</code> , <code>.GKr</code> , <code>.GKs</code> , <code>.Gto</code> , <code>.GbCa</code> , <code>.GbNa</code> , <code>.GpCa</code> , <code>.GpK</code> , <code>.PCa</code> , <code>.PNaK</code>	TTP06 scalable conductances
<code>m.V_rest</code>	<i>property</i> — resting potential, mV
<code>m.name</code>	<i>property</i> — model name
<code>m.n_states</code>	<i>property</i> — state count (18 for TTP06)
<code>m.state_names</code>	<i>property</i> — ordered ionic state names
<code>m.gate_indices</code> , <code>m.concentration_indices</code>	<i>property</i> — state-column indices by kind
<code>m.use_epi_ito_kinetics</code>	<i>property</i> — cell-type Ito switch

Methods

Call	Does
<code>m.compute_Iion(V, ionic_states)</code>	total ionic current
<code>m.compute_gate_steady_states(V, ionic_states)</code>	gate x_{∞}
<code>m.compute_gate_time_constants(V, ionic_states)</code>	gate τ
<code>m.compute_concentration_rates(V, ionic_states)</code>	concentration derivatives
<code>m.get_initial_state(n_cells=1)</code>	build the resting state tensor
<code>m.step(V, ionic_states, dt, I_stim=None)</code>	advance one Rush–Larsen ionic step
<code>m.run(t_end, dt=0.01, stim_times=None, stim_duration=1.0, stim_amplitude=-80.0, save_interval=None)</code>	standalone 0-D run

Factories accept a **name or a pre-built instance**, so a scaled model can be passed straight in.

5. CardiacSimulation

```
sim = cc.monodomain(g, "ttp06", cond, stim) # fast, single potential – default
sim = cc.bidomain(g, "ttp06", cond, stim)  # when bath / edge loading matters
sim = cc.lbm(g, "ttp06", cond, stim, dt=0.005) # lattice-Boltzmann
```

Attributes (all properties)

Access	Meaning
<code>sim.Nx</code> , <code>sim.Ny</code>	grid size
<code>sim.dx</code> , <code>sim.dy</code>	spacing, cm
<code>sim.dt</code>	timestep, ms
<code>sim.t</code>	current time, ms
<code>sim.Vm</code> (alias <code>sim.V</code>)	live <code>(Nx, Ny)</code> voltage, mV
<code>sim.phi_e</code>	extracellular potential — bidomain only
<code>sim.mask</code>	tissue mask
<code>sim.engine_type</code>	<code>"monodomain"</code> / <code>"bidomain"</code> / <code>"lbm"</code>
<code>sim.ionic_model</code>	the live model instance
<code>sim.ionic_states</code>	live <code>(n_dof, n_states)</code> tensor
<code>sim.state_names</code>	the ionic state names
<code>sim.Cm</code>	capacitance, pF/cm²
<code>sim.boundary_mode</code>	the stencil ghost rule

Methods — run & control

Call	Does
<code>sim.run(t_end, save_every=1.0, *, batch=None, record="Vm", callback=None)</code>	run it — eager → <code>SimulationResult</code> ; <code>batch=k</code> → iterator of chunks
<code>sim.snapshots(t_end, save_every=1.0, *, record="Vm", callback=None)</code>	yield <code>SimulationSnapshot</code> frames lazily
<code>sim.step()</code>	advance one timestep
<code>sim.reset()</code>	rebuild at t=0
<code>sim.with_(*overrides)</code>	return a NEW sim with new knobs — sweep-safe, original untouched

Methods — stimulus & clamp

Call	Does
<code>sim.stimulate(region, start_time=0.0, duration=1.0, amplitude=-52.0)</code>	append a stimulus (<code>Stim</code> , callable, or mask) and rebuild from t=0
<code>sim.add_stimulus(mask, start_time, duration, amplitude=-80.0)</code>	same, mask-form alias
<code>sim.add_pacing(mask, bcl=1000.0, n_beats=10, start_time=0.0, duration=2.0, amplitude=-80.0)</code>	add a pacing train
<code>sim.clamp_voltage(mask, voltage, start_time=None, duration=None)</code>	hold V (scalar, field, or <code>callable(t)</code>)
<code>sim.add_clamp_protocol(mask, steps, start_time=0.0)</code>	multi-step VC — <code>steps=[(mV, ms), ...]</code>
<code>sim.release_clamp()</code>	remove the clamp
<code>sim.inject_current(mask, amplitude)</code>	inject current mid-run

Methods — heterogeneity

Call	Does
<code>sim.scale_conductance(current_name, factor, mask=None)</code>	drug block / upregulation; compounds, rebuilds from t=0
<code>sim.set_conductivity(mask, D)</code>	set a region's diffusivity (scar: <code>D=0.0</code>)
<code>sim.scale_conductivity(mask, factor)</code>	multiply a region's diffusivity
<code>sim.set_parameter(name, value, mask=None)</code>	raises <code>NotImplementedError</code>
<code>sim.get_parameter_field(name)</code>	raises <code>NotImplementedError</code>

Methods — state injection (mono/bidomain only; LBM raises)

Call	Does
<code>sim.get_state(name)</code>	read one ionic state as <code>(Nx, Ny)</code>
<code>sim.set_state(name, values)</code>	overwrite one ionic state
<code>sim.set_voltage(V)</code>	overwrite the membrane voltage

Methods — probes & shortcuts

Call	Does
<code>sim.add_probe(name, x, y)</code>	record a trace at an <code>(x, y)</code> cm point
<code>sim.get_traces()</code>	return the recorded probe traces
<code>sim.clear_traces()</code>	discard recorded traces
<code>sim.compute_cv(x1, x2, y, threshold=-20.0)</code>	conduction velocity between two x-indices
<code>sim.compute_apd(x, y, repol=0.9)</code>	APD at a point
<code>sim.compute_activation_time(threshold=-20.0)</code>	activation-time map

6. `SimulationResult`

```
r = sim.run(t_end=40.0, save_every=0.5)
```

Attributes

Access	Meaning
<code>r.times</code>	<code>(T,)</code> ms
<code>r.Vm</code> (alias property <code>r.V</code>)	<code>(T, Nx, Ny)</code> voltage — masked-out nodes are NaN
<code>r.phi_e</code>	<code>(T, Nx, Ny)</code> , or <code>None</code>
<code>r.dx</code> , <code>r.dy</code>	spacing carried from the sim
<code>r.ionic_states</code>	present if <code>record=("Vm","ionic_states")</code>
<code>r.domain_mask</code>	tissue mask
<code>r.boundary_mode</code> , <code>r.Cm</code> , <code>r.chi</code>	analysis context
<code>r.conductivity</code> , <code>r.ionic_model</code> , <code>r.cell_type</code>	provenance
<code>r.fields</code>	<i>cached property</i> — the <code>Fields</code> accessor (§7)

Methods

Call	Does
<code>r.cv(x1, x2, y, **kw)</code>	conduction velocity cm/s between two x-indices
<code>r.cv_between(p1, p2, **kw)</code>	CV along the line between two <code>(ix,iy)</code> nodes
<code>r.radial_cv(center, **kw)</code>	outward CV map from a point source
<code>r.apd(**kw)</code>	APD map
<code>r.apd_per_beat(ix, iy, **kw)</code>	APD of each beat at a node
<code>r.lat(**kw)</code>	<code>(Nx, Ny)</code> local activation time map
<code>r.df_map()</code>	dominant-frequency map
<code>r.restitution(ix, iy, **kw)</code>	<code>(DI, APD)</code> restitution curve
<code>r.restitution_slope(ix, iy, **kw)</code>	max slope + DI* (alternans onset)
<code>r.video(slug, **kw)</code>	render to a convention <code>media/</code> path

7. Fields and VectorField

Fields **attributes** — reached as `r.fields.<name>` , all properties, computed lazily.

Access	Meaning
<code>.voltage_gradient</code>	∇V
<code>.voltage_flux</code>	voltage flux
<code>.current_flux</code>	current flux
<code>.electric_field</code>	$E = -\nabla V$
<code>.velocity</code>	wavefront velocity
<code>.speed</code>	velocity
<code>.direction</code>	propagation direction
<code>.divergence</code>	$\nabla \cdot F$
<code>.curvature</code>	wavefront curvature
<code>.vorticity</code>	$\nabla \times v$
<code>.source_sink</code>	source–sink balance
<code>.quality</code>	per-node reliability of the derived fields
<code>.mask</code>	the tissue mask used
<code>.derivatives</code>	the raw derivative sub-accessor
<code>.integrals</code>	the integral sub-accessor (<code>region_integral</code> , <code>net_flux</code> , <code>circulation</code>)

Methods — none.

VectorField **attributes** — returned by the vector-valued fields above.

Access	Meaning
<code>v.x</code> , <code>v.y</code>	components
<code>v.components</code>	both, stacked
<code>v.magnitude</code>	length per node
<code>v.angle</code>	direction, radians
<code>v.shape</code> , <code>v.device</code> , <code>v.dtype</code>	tensor metadata

Methods — none.

8. SingleCellResult

```
sc = cc.single_cell("ttp06", celltype="EPI", pre_pace=5)
```

Attributes

Access	Meaning
sc.times	(T,) ms
sc.V	(T,) voltage trace, mV
sc.final_state	end ionic state — carry into a tissue run
sc.model	the model name
sc.dt, sc.Cm	timestep, capacitance
sc.v_peak	property — overshoot, mV
sc.v_rest	property — diastolic rest, mV

Methods

Call	Does
sc.apd(repol=0.9, threshold=-20.0)	action potential duration, ms

Full signature:

```
single_cell(model="ttp06", *, celltype="ENDO", dt=None, bcl=1000.0, n_beats=1, pre_pace=0, stim_amplitude=-52.0, stim_duration=2.0, t0=10.0, Cm=1.0, save_e
```

9. Video, Gradient, VideoInfo

```
v = cc.Video(r, field="Vm", gradient=cc.Gradient(cmap="inferno"))
info = cc.render(v, "spiral-wave") # -> media/{question}/videos/{date}/spiral-wave_01.mp4
```

Video

Constructors

Call	Does
cc.Video.bare(data, **kw)	no chrome — raw raster
cc.Video.annotated(data, **kw)	with title / colourbar / time

Attributes

Access	Meaning
v.data	the result (or array) to draw
v.field	which field — "Vm", "phi_e", or an array
v.gradient	the Gradient colour spec
v.label	panel label
v.front	front-contour level, or None
v.isochrones	draw activation isochrones
v.mask	tissue mask
v.style, v.aspect, v.units	chrome style, aspect ratio, unit display

Methods

Call	Does
v.preview(t_ms=None, *, frame=None, slug="preview", question="lab", bulk=True, **kw)	render ONE still
v.display_values(t)	return the (Nx, Ny) array actually drawn at frame t
v.masked_iter(idx)	iterate masked frames
v.requires_figure()	report whether it needs a matplotlib figure vs a raw raster

Gradient

Constructors

Call	Does
<code>cc.Gradient.physiological(**kw)</code>	fixed physiological mV range (default)
<code>cc.Gradient.autoscale(**kw)</code>	scale to the data
<code>cc.Gradient.diverging(**kw)</code>	diverging map, for signed fields
<code>cc.Gradient.rest_anchored(vmax=40.0, **kw)</code>	anchor at rest, cap at <code>vmax</code>
<code>cc.Gradient.zoom(span=8.0, below=0.3, **kw)</code>	narrow window — reveals sub-threshold structure

Attributes

Access	Meaning
<code>gr.cmap</code>	colormap name / list / <code>Colormap</code>
<code>gr.value_range</code>	"physiological" or an explicit <code>(vmin, vmax)</code>
<code>gr.gamma</code>	gamma correction
<code>gr.levels</code>	discrete contour levels, or <code>None</code>
<code>gr.bad</code>	colour for NaN / masked nodes
<code>gr.interpolation</code>	raster interpolation
<code>gr.v_rest</code> , <code>gr.rest_vmax</code>	rest anchor and cap
<code>gr.zoom_span</code> , <code>gr.zoom_below</code>	zoom-window width and lower offset

Methods

Call	Does
<code>gr.key()</code>	return a cache key for this spec
<code>gr.resolve(masked_values, *, field="Vm")</code>	resolve to a concrete norm + colormap

VideoInfo

A str-like path returned by `render`.

Attributes

Access	Meaning
<code>info.path</code>	the written file
<code>info.n_frames</code> , <code>info.fps</code> , <code>info.duration_s</code>	timing
<code>info.backend</code> , <code>info.codec</code>	encoder actually used
<code>info.width</code> , <code>info.height</code>	pixels
<code>info.vmin</code> , <code>info.vmax</code>	colour range used
<code>info.stride</code> , <code>info.size_bytes</code> , <code>info.bitrate</code>	frame stride, file size, bitrate

Methods — none.

`render(video_or_list, slug, *, question="lab", fps=20.0, max_frames=300, format="mp4", ...)` — pass a **list** of `Video` s for side-by-side panels.

10. CardiacMeshData

```
mesh = cc.create_cardiac_mesh(Lx, Ly, dx, D=1.4, chi=1400.0) # lengths first, THEN spacing (cm)
cc.save_cardiac_mesh(path, mesh); mesh = cc.load_cardiac_mesh(path)
sim = cc.monodomain(mesh)
```

Attributes

Access	Meaning
<code>mesh.dx</code> , <code>mesh.dy</code>	spacing, cm
<code>mesh.mask</code>	tissue mask
<code>mesh.D_xx</code> , <code>mesh.D_yy</code> , <code>mesh.D_xy</code>	per-node diffusivity tensor (raw — pre χ ·Cm)
<code>mesh.chi</code> , <code>mesh.Cm</code>	cm⁻¹ , µF/cm²
<code>mesh.ionic_model</code>	model NAME (a string)
<code>mesh.dt</code>	timestep, ms
<code>mesh.stimuli</code>	list of normalized stimulus dicts
<code>mesh.sigma_i</code> , <code>mesh.sigma_e</code>	bidomain conductivity field tuples
<code>mesh.group_labels</code> , <code>mesh.group_cell_types</code>	region tags and their cell types
<code>mesh.boundary</code>	"insulated" / "bath"

Methods — none.

Argument-order trap. `Grid(Nx, Ny, dx)` leads with **node counts**; `create_cardiac_mesh(Lx, Ly, dx)` leads with **lengths in cm**. Both end in `dx`. Tied by `L = (N-1)·dx`.

11. **Distribution** and **SimulationSnapshot**

Distribution

`cc.Distribution(kind, **kwargs)` — a per-node random parameter spec, accepted by `set_conductivity` / `scale_conductivity`.

Attributes — `d.kind` and the kind-specific kwargs.

Methods

Call	Does
<code>d.sample(shape, device="cpu", dtype=torch.float64)</code>	draw a <code>shape</code> tensor of values

SimulationSnapshot

One frame yielded by `sim.snapshots(...)`.

Attributes

Access	Meaning
<code>snap.t</code>	frame time, ms
<code>snap.Vm</code> (alias property <code>snap.V</code>)	<code>(Nx, Ny)</code> voltage
<code>snap.phi_e</code>	<code>(Nx, Ny)</code> , or <code>None</code>
<code>snap.Nx</code> , <code>snap.Ny</code> , <code>snap.dx</code> , <code>snap.dy</code>	geometry
<code>snap.ionic_states</code>	if requested via <code>record=</code>

Methods — none.

Free functions

- Analysis** — `activation_time` `max_dvdt_time` `conduction_velocity` `cv_between` `radial_cv` `apd_at` `apd_map` `apd_per_beat` `restitution_curve` `restitution_slope` `wavelength` `di` `dominant_frequency` `dominant_frequency_map` `phase_map` `phase_singularities` `wavefront_mask`
- Geometry masks** — `circle_mask` `rectangle_mask` `annulus_mask` `left_edge_mask` `right_edge_mask` `top_edge_mask` `bottom_edge_mask` `point_distance` `boundary_distance` `fiber_field_uniform` `fiber_field_transmural`
- Protocols (these RUN simulations)** — `erp(...)` · `erp_proxy(apd90)` · `post_repol_refractoriness(erp, apd90)`
- 0-D / safety** — `single_cell(...)` · `safety_factor(r, ...)` · `threshold_charge(...)`
- One-shot runners** — `simulate(mesh, t_end, ..., engine=...)` · `run_monodomain` / `run_bidomain` / `run_lbm`
- I/O** — `save_result` · `load_result` · `save_cardiac_mesh` · `load_cardiac_mesh` · `create_cardiac_mesh`
- Figures & video** — `propagation_video` · `apd_map_figure` · `activation_isochrones` · `render` / `render_video`

Units

Quantity	Where	Unit
Spacing / lengths	<code>dx</code> , <code>dy</code> , <code>Lx</code> , <code>Ly</code> , <code>stim width</code> / <code>radius</code> , <code>point (x,y)</code>	cm
Node counts	<code>Nx</code> , <code>Ny</code>	integer count
Time	<code>dt</code> , <code>start_time</code> , <code>duration</code> , <code>t_end</code> , <code>save_every</code> , <code>bcl</code>	ms
Voltage	<code>Vm</code> , <code>clamp=</code>	mV
Stimulus current	<code>amplitude</code>	µA/pF (negative = depolarizing)
Diffusivity	<code>D_eff</code>	cm²/ms
Conductivity	<code>sigma_i</code> , <code>sigma_e</code> , <code>sigma_eff</code>	mS/cm
Surface-to-volume	<code>chi</code>	cm⁻¹
Capacitance	<code>Cm</code>	µF/cm²
Conduction velocity	<code>r.cv()</code>	cm/s
Fiber angle	<code>fiber_angle</code>	radians

`float64` on CPU by default; pass `device="cuda"` to the factories. The **Optimizer** is the only subsystem that speaks millimetres (`dx_mm`, `domain_mm`) and converts at its edge.